



AEROGO

Real-Time Flight Alert System

Author	Enejo Unwuchola
Version	v1.0
Date	April 2026
Status	Draft

1. Executive Summary

AeroGo is an event-driven flight tracking and alert system built with Apache Kafka and Go. It processes continuous flight data streams, detects meaningful state changes, and notifies subscribed users via SMS with low latency.

Problem

Users lack real-time visibility into flight state changes such as delays and gate reassignments.

Solution

An event-driven pipeline that ingests live flight data, detects meaningful changes, and sends SMS alerts instantly.

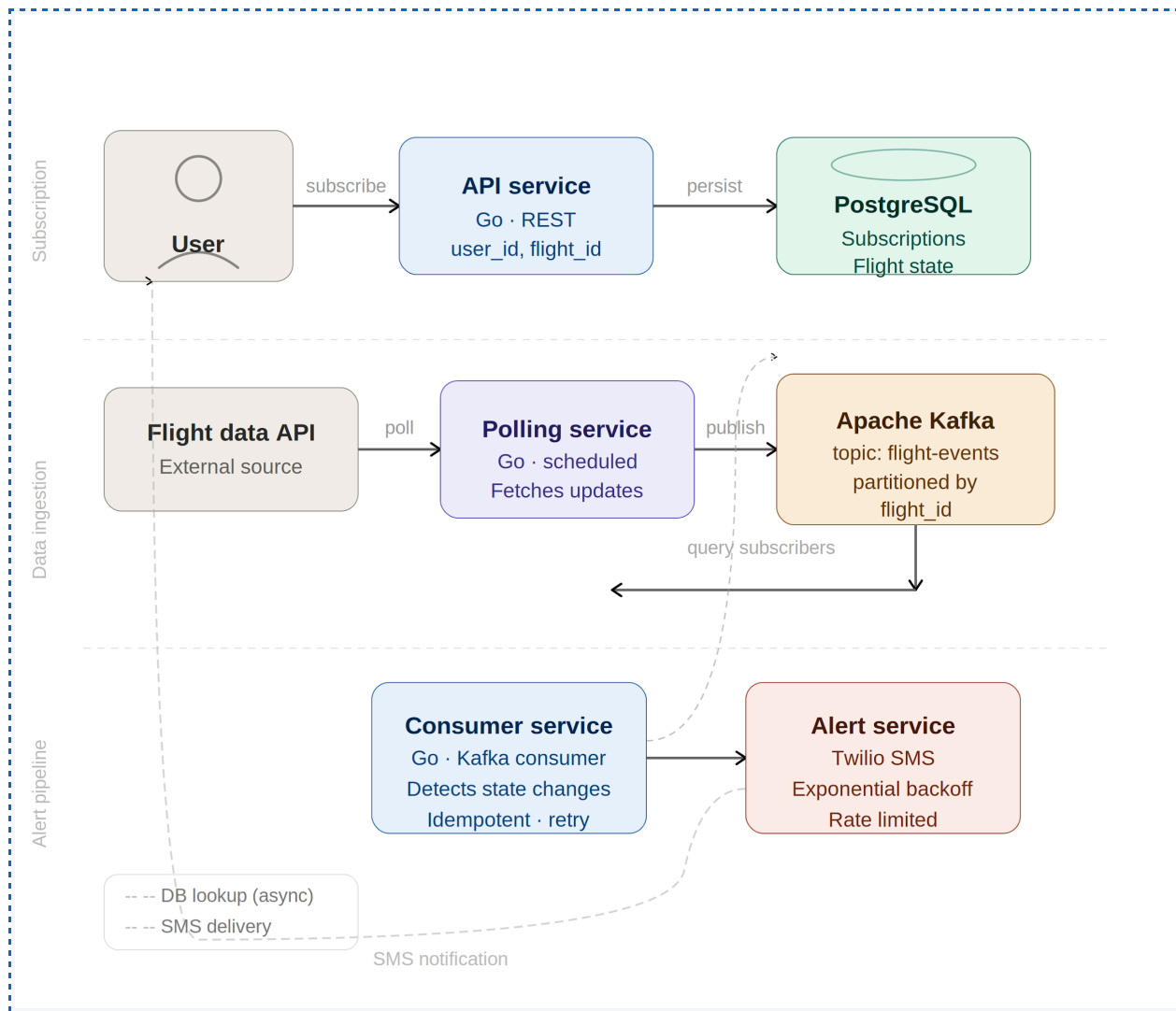
Outcome

Yet to be added as progress continues.

2. System Overview

AeroGo allows users to subscribe to specific flights and automatically receive SMS notifications when the system detects meaningful transitions in flight state — including delay increases, gate changes, and departure events.

2.1 Architecture Diagram



2.2 High-Level Data Flow

The following steps describe the end-to-end processing pipeline:

- User subscribes to a flight via `user_id` and `flight_id`
- Polling service continuously fetches updates from an external flight API
- Each update is published to a Kafka topic, partitioned by `flight_id`
- A Go-based consumer reads the ordered event stream
- Incoming state is compared against previously stored state
- On detecting a meaningful change, the system queries tracking users
- SMS alerts are sent to subscribers via Twilio

3. Technology Stack

Component	Role in System
Go	Core application language — consumer, business logic, and alert dispatch
Apache Kafka	Event streaming backbone — ordered, durable flight update log partitioned by flight_id
PostgreSQL	Persistent storage — user subscriptions and flight state snapshots
Twilio	SMS delivery layer for user-facing flight change notifications

4. Key Features & Design Decisions

4.1 Idempotent Processing

Kafka's at-least-once delivery guarantee means the same event can be processed more than once. AeroGo handles this through idempotent processing logic that prevents duplicate SMS alerts from being sent to users.

[*Diagram or code snippet: Idempotency key design / deduplication mechanism*]

4.2 Retry Logic with Exponential Backoff

Failed SMS deliveries are retried automatically using exponential backoff to avoid thundering-herd issues during transient Twilio failures.

[*Diagram or code snippet: Retry flowchart or backoff configuration*]

4.3 Rate Limiting / Cooldown Windows

To prevent notification spam during rapid flight update bursts, AeroGo applies configurable cooldown windows between consecutive alerts for the same flight-user pair.

[*Diagram or timeline: Cooldown window behavior example*]

5. Performance & Metrics

Insert benchmarks, latency measurements, or throughput figures relevant to the system's performance during testing or production.

[Metrics table or chart — e.g., avg alert latency, Kafka throughput, retry rates]

Metric	Target	Actual
End-to-end alert latency	< 500ms	Yet to be added as progress continues.
Kafka consumer throughput	Yet to be added as progress continues.	Yet to be added as progress continues.
SMS delivery success rate	> 99%	Yet to be added as progress continues.

6. Challenges & Solutions

Document key technical challenges encountered during development and how they were resolved.

Challenge	Impact	Solution
Duplicate Kafka events	Duplicate SMS alerts	Idempotency keys + deduplication store
Yet to be added as progress continues.	Yet to be added as progress continues.	Yet to be added as progress continues.

7. Future Work

Planned improvements and potential extensions to the system:

- Multi-channel notifications (email, push notifications)
- User-facing dashboard for flight subscription management
- Kafka consumer group scaling for higher throughput
- Dead-letter queue for persistent SMS failure handling
- Yet to be added as progress continues.

8. Conclusion

Yet to be added as progress continues.

References

- Apache Kafka Documentation — <https://kafka.apache.org/documentation/>

- Twilio SMS API — <https://www.twilio.com/docs/sms>
- Go Documentation — <https://go.dev/doc/>
- Yet to be added as progress continues.